asyncio.gather 학습 노트

날짜: 2026-05-22
복습 예정: D+1, D+3, D+7

한 줄 요약

여러 작업을 동시에 시작하고, 모두 끝날 때까지 기다리는 함수.

---

1. 비유

음식 배달에 비유:

asyncio.gather 없이 (순차):

```
짜장면 주문 -> 기다림(10분) -> 탕수육 주문 -> 기다림(8분) = 총 18분
```

asyncio.gather 있이 (병렬):

```
짜장면 + 탕수육 동시 주문 -> 기다림(10분) = 총 10분
```

Dev 2에서:

```
[python]
# 정적 분석(30초) + 보안 스캔(30초)을 동시에 실행 -> 총 30초
# 순서대로 하면 60초 걸릴 것을 절반으로 단축
```

---

2. 개념

> [asyncio.gather()]는 여러 비동기 함수(coroutine)를 동시에 실행하고,
> 모든 결과를 리스트로 반환한다.

동기 vs 비동기:

구분	설명	키워드
동기(sync)	하나 끝나야 다음 시작	일반 함수 [def]
비동기(async)	동시에 여러 개 실행 가능	[async def], [await]

---

3. 왜 쓰나

- 없으면: Pylint 실행(30초) 끝난 후 Bandit 실행(30초) -> 총 60초
- 있으면: 둘 동시 실행 -> 총 30초 -> PRD "60초 이내" 목표 달성

---

4. 코드 (Dev 2)

```
[python]
import asyncio

# 기본 사용법
results = await asyncio.gather(
    run_static_analysis(payload),   # 동시 시작
    run_security_scan(payload),      # 동시 시작
)
# 둘 다 끝나면 results = [static_result, security_result]

# return_exceptions=True: 하나가 실패해도 나머지 결과 반환
results = await asyncio.gather(
    run_static_analysis(payload),
    run_security_scan(payload),
    return_exceptions=True   # <- 이게 없으면 하나 실패 시 전체 실패
)

# 실패한 것 걸러내기
valid = [r for r in results if not isinstance(r, Exception)]
```

주의사항:

```
[python]
# async def가 아닌 일반 함수는 gather에 넣을 수 없음
async def run_static_analysis(payload):  # async 필수
    ...

# await 없이 호출하면 실행 안 됨
result = run_static_analysis(payload)         # [X] 실행 안 됨
result = await run_static_analysis(payload)  # [OK] 올바른 사용
```

복습 퀴즈

1. Q: asyncio.gather와 순차 실행의 차이는?
   A: (gather는 동시 실행 -> 시간 절약. 순차는 하나씩 -> 느림)

2. Q: [return_exceptions=True]를 쓰는 이유는?
   A: (하나의 에이전트가 실패해도 나머지 결과는 정상 반환하기 위해)

3. Q: [async def]와 [def]의 차이는?
   A: (async def는 비동기 함수로 await로 호출해야 함. def는 동기 함수)